

Implementação do Pré-Processamento de Dados:

Dendê Softhouse

Eli Barreto, Elias Barreto, Emerson Lucas, Kaio Pinheiro

Centro Universitário de Excelência (UNEX)

Av. Artêmia Pires Freitas, s/n - Sim, Feira de Santana - BA, 44085-370

eli.barreto@aluno.unex.edu.br, 241030866@aluno.unex.edu.br,
elias.barreto@aluno.unex.edu.br, kaio.pinheiro@aluno.unex.edu.br.

Abstract. *Data quality is a critical factor for the effectiveness of Machine Learning algorithms. This report documents the second phase of the project developed by Dendê Softhouse, focused on building a Python library from scratch, without external dependencies (such as Pandas or Scikit-Learn). Following the consolidation of the Exploratory Data Analysis (EDA) module, the current phase presents the Preprocessing module, which includes missing values handling, numerical scaling, and categorical variable encoding. The results demonstrate the architecture's robustness through rigorous unit testing and validation on a real-world dataset from the Spotify ecosystem.*

Resumo. *A qualidade dos dados é um fator crítico para a eficácia de algoritmos de Aprendizado de Máquina. O presente relatório documenta a segunda fase do projeto desenvolvido pela Dendê Softhouse, focado na construção de uma biblioteca em linguagem Python, construída inteiramente do zero, sem dependências de pacotes externos consolidados (como Pandas ou Scikit-Learn). Após a consolidação do módulo de Análise Exploratória de Dados (EDA), a fase atual apresenta o módulo de Pré-processamento, que engloba o tratamento de valores ausentes, escalonamento numérico e codificação de variáveis categóricas. Os resultados demonstram a robustez da arquitetura através de testes unitários rigorosos e validação em um conjunto de dados real do ecossistema Spotify.*

1. Introdução

No paradigma contemporâneo da Ciência de Dados, a fase de preparação e estruturação das bases de informações consome, historicamente, a maior parcela de tempo dos profissionais da área. Dados brutos no mundo real são inerentemente ruidosos, incompletos e heterogêneos. Tentar extrair estatísticas descritivas ou treinar algoritmos sobre dados inconsistentes gera o problema conhecido como **Garbage In, Garbage Out** (GIGO).

A **Dendê Softhouse**, motivada pela necessidade de não apenas aplicar ferramentas prontas, mas de compreender a profunda matemática computacional por trás do processamento, iniciou o desenvolvimento de uma suíte própria e nativa em Python. Na primeira fase deste projeto, foi concebido o módulo de Análise Exploratória de Dados (EDA), através da classe `Statistics`, responsável por extrair medidas de tendência central, dispersão e frequências.

O objetivo da fase atual, documentada neste relatório, é expandir este ecossistema solucionando o problema da qualidade dos dados. Foi construído um módulo de Pré-processamento capaz de lidar com a ausência de dados, nivelamento de grandezas numéricas e conversão de características categóricas, garantindo uma estrutura de dicionários sanitizada e pronta para consumo analítico.

2. Descrição do Dataset e sua Relevância

Para atestar a robustez do código contra o estresse estrutural de uma base não-homogênea, o desenvolvimento foi validado utilizando o conjunto de dados **Spotify Data Clean**.

Este dataset é composto por **8.582 registros musicais e 15 atributos** (colunas), contendo informações como a popularidade da faixa, o número de seguidores do artista, a presença de conteúdo explícito, o gênero musical e a duração das músicas.

A escolha desta base é altamente relevante por três motivos técnicos que simulam o cenário ideal para o pré-processamento:

- 2.1. Presença de Nulos:** As colunas `artist_name` e `artist_genres` possuem, respectivamente, 3 e 3.361 registros vazios (missing values), exigindo imputação para não inviabilizar a análise.
- 2.2. Discrepância de Escalas:** A coluna `track_duration_min` opera em decimais (ex: 2.55 minutos), enquanto `artist_followers` chega à casa dos milhões, exigindo nivelamento matemático.
- 2.3. Variáveis Categóricas e Binárias:** Colunas como `album_type` (álbum, single, compilação) e `explicit` (True/False) exigem tradução para representações numéricas computáveis.

3. Descrição do Código e Métodos Implementados

A arquitetura do projeto foi desenhada sob o paradigma de Programação Orientada a Objetos (POO), onde a classe `Preprocessing` orchestra subprocessadores especializados. A grande inovação desta fase é a integração direta com o módulo de Análise Exploratória (classe `Statistics`), permitindo cálculos dinâmicos em tempo de execução.

Abaixo, detalha-se a implementação e a aplicação prática de cada módulo utilizando o *dataset* Spotify Data Clean.

3.1. Módulo MissingValueProcessor (Tratamento de Dados Ausentes)

Este subprocessador higieniza lacunas no *dataset*. Os métodos *isna()* e *notna()* filtram registros corrompidos (representados por *None*), enquanto o *dropna()* os exclui. O método mais crítico, no entanto, é o *fillna()*, que realiza a imputação dinâmica de dados, evitando a perda de linhas valiosas.

```
def fillna(self, columns: Set[str] = None, value: Any = 0) -> Dict[str, List[Any]]: 1 usage  Elias Barreto +1
    """
    Preenche valores nulos (None) nas colunas especificadas com um valor fixo (Any = 0).
    Modifica o dataset da classe.
    """
    alvos = self._get_target_columns(columns)

    for col in alvos:
        dados = self.dataset[col]
        for i in range(len(dados)):
            if dados[i] in [None, "", "N/A"]:
                dados[i] = value

    return self.dataset
```

Aplicação Prática no Spotify: A coluna *artist_genres* possuía 3.361 registros nulos.

- **Antes:** ["moombahton", *None*, "dark r&b", *None*...]
- **Implementação/Uso:**
prep.fillna(columns={"artist_genres"}, value="Desconhecido")
- **Depois:** ["moombahton", "Desconhecido", "dark r&b", "Desconhecido"...]

3.2. Módulo Scaler (Nivelamento Numérico)

Responsável por harmonizar grandezas numéricas discrepantes, evitando viés analítico em modelos preditivos. O método *MinMax_scaler* aplica a fórmula

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}, \text{ onde:}$$

X_{norm} : É o resultado final. O seu novo valor normalizado (que será um número entre 0 e 1).

X : É o valor original atual que você está analisando (ex: a popularidade de uma música específica).

X_{min} : É o **menor** valor absoluto que existe em toda a sua coluna.

X_{max} : É o **maior** valor absoluto que existe em toda a sua coluna.

$X_{max} - X_{min}$: É o que chamamos de **Amplitude**. É o tamanho total do "espaço" entre o menor e o maior número da sua lista.

Trecho da Implementação: O código abaixo demonstra a lógica nativa construída para o cálculo da amplitude e normalização individual iterando sobre a lista de dados:

```
def minMax_scaler(self, columns: Set[str] = None) -> Dict[str, List[Any]]: 1 usage 2 Elias Barreto +1
    """
    Aplica a normalização Min-Max ( $X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$ )
    nas colunas especificadas. Modifica o dataset.
    """
    alvos = self._get_target_columns(columns)

    for col in alvos:
        dados = self.dataset[col]
        val_min = min(dados)
        val_max = max(dados)
        amplitude = val_max - val_min

        if amplitude == 0:
            self.dataset[col] = [0.0 for _ in dados]
            continue # Pula para a próxima coluna

        dados_escalados = []
        for x in dados:
            x_normalizado = (x - val_min) / amplitude
            dados_escalados.append(x_normalizado)

        self.dataset[col] = dados_escalados

    return self.dataset
```

- **Aplicação Prática no Spotify:** A coluna `artist_popularity` possui valores absolutos espalhados em uma escala larga.

Antes: [77, 64, 48, 0, 85...] (Diferença brusca de grandezas).

Uso: `prep.scale(columns={"artist_popularity"}, method="minMax")`

Depois: [0.90, 0.75, 0.56, 0.0, 1.0...] (Dados espremidos proporcionalmente entre 0 e 1, prontos para cálculos de distância geométrica).

O módulo também conta com o `standard_scaler` (Z-Score), que prova a integração do ecossistema ao instanciar a classe `Statistics` nativa (`stats = Statistics(self.dataset)`) para extrair a Média e o Desvio Padrão antes de aplicar a padronização.

3.3. Módulo Encoder (Codificação de Textos)

Permite a tradução de palavras para matrizes matemáticas, viabilizando o cálculo algébrico sobre dados nominais.

- O `label_encode()` mapeia strings únicas para inteiros ordinais sequenciais.

- O `oneHot_encode()` transforma uma coluna com N categorias em N novas colunas booleanas isoladas. A implementação utiliza formatação dinâmica de *strings* no Python (`f'{col}_{categoria}'`) para injetar o nome da categoria no cabeçalho numérico, removendo a coluna original no processo.

```
def label_encode(self, columns: Set[str]) -> Dict[str, List[Any]]: 1 usage  Elias Barreto +1
    """
    Converte cada categoria em uma coluna em um número inteiro.
    Modifica o dataset.
    """
    for col in columns:
        dados = self.dataset[col]
        categoriasUnicas = list(set(dados))
        categoriasUnicas.sort()

        mapaCategorias = {}
        for i, categoria in enumerate(categoriasUnicas):
            mapaCategorias[categoria] = i

        dadosCodificados = []
        for valor in dados:
            dadosCodificados.append(mapaCategorias[valor])

        self.dataset[col] = dadosCodificados

    return self.dataset
```

- **Aplicação Prática no Spotify:** O algoritmo não compreende texto categórico. A coluna `album_type` precisa ser binarizada.

Antes (Coluna Única): `album_type`: ["album", "single", "compilation"]

Uso: `prep.encode(columns={"album_type"}, method="oneHot")`

Depois (Múltiplas Colunas Booleanas):

- `album_type_album`: [1, 0, 0]
- `album_type_single`: [0, 1, 0]
- `album_type_compilation`: [0, 0, 1]

4. Conclusão e Impacto na Análise Descritiva

O desenvolvimento do motor de pré-processamento nativo cumpriu plenamente os objetivos da Dendê Softhouse, entregando uma ferramenta robusta e altamente tipada, validada com sucesso através de rigorosos Testes Unitários (unittest).

A integração destas etapas impacta de maneira profunda e transformadora a qualidade da **Análise Descritiva e Exploratória (EDA)**. Sem o uso prévio do `MissingValueProcessor`, a extração de métricas de tendência central (como as médias

calculadas pela classe `Statistics`) seria fatalmente interrompida por exceções de tipo `(TypeError)` decorrentes de *NoneTypes* somados a inteiros.

Adicionalmente, a aplicação dos `Scalers` viabiliza uma leitura de variância e covariância limpa, isolando o real comportamento da distribuição dos dados em detrimento da diferença ingênua de grandezas, enquanto os `Encoders` permitem, pela primeira vez, que variáveis textuais cruciais — como o gênero do artista no caso do Spotify — sejam correlacionadas matematicamente na análise descritiva. Deste modo, a base de dados encontra-se não apenas descritível, mas pronta para ingestão futura em algoritmos de Inteligência Artificial.

5. Referências

- VANDERPLAS, Jake. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, 2016.
- SOCIEDADE BRASILEIRA DE COMPUTAÇÃO (SBC). *Template para Artigos e Relatórios*.
- PYTHON SOFTWARE FOUNDATION. *Python Language Reference, version 3.x*.
- Dendê Softhouse. *Documentação interna dos Módulos Statistics e Preprocessing*, 2024.